A Systematic Benchmark of Supervised and Unsupervised Graph Embedding Methods

Konstantina Koulaktsidou
Department of Electrical and Computer Engineering
National Technical University of Athens
email

Fragiska Kiminou
Department of Electrical and Computer Engineering
National Technical University of Athens
email

Nikolaos Moraitis
Department of Electrical and Computer Engineering
National Technical University of Athens
email

***Abstract*—Graph embedding techniques are important tools for representing graphs in low-dimensional vector spaces, enabling effective downstream tasks. In this work, we evaluate unsupervised and supervised representative whole graph embedding techniques on real world benchmark datasets. Specifically, we compare Graph2Vec, NetLSD, and the Graph Isomorphism Network (GIN) across three datasets (MUTAG, ENZYMES, and IMDB-MULTI). Performance is assessed through graph classification and clustering tasks, considering predictive accuracy, computational efficiency and scalability. Our expertiments indicate that GIN achieves the highest accuracy but requires substatial computational resources. Graph2Vec balances performance and efficiency, while NetLSD provides competitive clustering performance with minimal overhead. Therefore it is apparent that each method has its distinct trade-offs. All methods maintain reasonable robustness under moderate perturbations, with sensitivity depending on dataset characterestics. These findings provide practical insights that aid in selecting graph embedding approaches based on the task requirements, dataset properties and computationional constraints.**

***Index Terms*—Graph2Vec, NetLSD, GIN, Graph Embedding, Graph Classification, Graph Clustering**

I. Introduction

Graphs provide a powerful representation for complex relational data and are widely used in domains such as network analysis, bioinformatics and chemistry. In many scenarios, the learning task is defined at the graph level, such as classifying molecular graphs by biological activity. However, graphs have irregular structure with variable sizes, unordered nodes and complex connectivity patterns, features that make them face challenges to traditional machine learning algorithms, which typically require fixed dimensional vector inputs.

Graph embedding methods address this challenge by mapping entire graphs into continuous vector representations that preserve the structural and semantic information of the graph. These kinds of representations enable the use of standard machine learning techniques for downstream tasks such as graph classification, clustering and similarity search. As a result, graph embeddings have successfully become a core component of modern graph based learning pipelines.

Early approaches relied on handcrafted features or graph kernels, which often suffer from high computational complexity or limited expressiveness [CITATIONS NA VROYME]. Recent approaches automatically learn graph representations and can be categorized into unsupervised[CITATIONS] and supervised techniques[CITATIONS]. Unsupervised methods learn graph representations without relying on labeled data exploiting often structural patterns. In this category belong Graph2Vec [CITATION], which adapts the skip-gram model to learn graph level embeddings from rooted substructures, and NetLSD, which uses heat kernel signatures derived from the graph Laplacian to capture global topological characteristics. On the other hand, supervised procedures, which are based on graph neural networks (GNNS) [CITAITON], learn task specific embeddings through end-to-end training. Among these, the Graph Isomorphism Network (GIN) [CITATION] has gained significant attention due to its strong theoretical expressiveness. GIN can learn highly discriminative graph representations while achieving superior performance on graph classification benchmarks. However, choosing this type of procedure typically comes with the cost of increased computational complexity, higher memory consumption and the need for labeled training data, which is often not provided to us.

Despite the development of graph embedding techniques[CITATIONS], comparisons between unsupervised and supervised methods remain limited[CITATIONS]. Many existing studies focus primarly on classification accuracy and overlooking significant considerations such as computational cost, embedding dimensionality and robustness to structural permutations. These are critical when working with real world small datasets (EMB1), where overfitting, instability and minimum training data can easily affect the model's performance.

In this work, we focus on EMB1 and we evaluate and compare unsupervised and supervised graph embedding methods on small benchmark datasets from the TUDataset collection: MUTAG, ENZYMES, and IMDB-MULTI. Studying small datasets is as meaningful as big ones, as they represent realistic scenarios in domains such as chemistry and biology, where labeled data is costly to obtain and sometimes no where to be found and model robustness is essential. From the unsupervised category, we use Graph2Vec and NetLSD and from the supervised GIN. To ensure fairness, all methods are evaluated using a unified experimental pipeline with minimal difference from GIN. The evaluation extends beyond standard classification and includes clustering behavior, computational efficiency and stability under random perturbations in the graphs structure.

The main contributions are summarized as follows:

- We present a comparative evaluation of unsupervised and supervised whole-graph embedding methods on widely used real-world datasets,
- We analyze the trade-off between classification accuracy and computational cost, highlighting practical considerations for method selection,
- We perform a stability analysis under structural permutations, assessing the robustness of different embedding paradigms

II. Related Work

III. Methods

This section describes the graph embedding methods evaluated in this work, along with the implementation details and experimental setup used to ensure fair comparison across models.

A. Graph2Vec

Graph2Vec [1], inspired by document representation techniques in natural language processing, represents each graph as a set of rooted subgraphs, which are treated like words are in a document. These subgraphs are extracted using the Weisfeiler Lehman (WL) relabeling process and in order to train the skipgram model it is being assigned a unique label for all the rooted subgraphs in the vocabulary. So the graph2vec's algorithm first generate those rooted subgraphs around every node in a given graph and then learns embeddings of the given graphs[CITE]. The Graph2Vec's algorithm is shown below:

Algorithm 1 Graph2Vec

Require: $\mathcal{G} = \{G_1, G_2, \ldots, G_{|\mathcal{G}|}\}$: set of graphs, where each $G_i = (N_i, E_i, \lambda_i)$
D: maximum WL depth
δ: embedding dimension
e: number of epochs
α: learning rate
Ensure: Graph embedding matrix $\Phi \in \mathbb{R}^{|\mathcal{G}| \times \delta}$
Initialize $\Phi \sim \mathcal{U}(-0.5/\delta, 0.5/\delta)$
for *epoch* $= 1$ to e **do**
    Shuffle $\mathcal{G}$
    for each graph $G_i \in \mathcal{G}$ **do**
        for each node $n \in N_i$ **do**
            for $d = 0$ to D **do**
                $sg \leftarrow$ GetWLSubgraph(n, G_i, d)
                Update Φ_i using Skip-Gram with learning rate α
            end for
        end for
    end for
end for
return Φ

Algorithm 2 GetWLSubgraph

Require: n: root node
$G = (N, E, \lambda)$: input graph
d: WL iteration depth
Ensure: Rooted subgraph label sg
$sg \leftarrow \lambda(n)$
for $i = 1$ to d **do**
    Collect multiset of neighbor labels of n
    Concatenate and hash labels
    Update sg
end for
return sg

For each graph, the multiset of rooted subgraphs obtained across WL iterations forms a *bagof subgraphs* representation, analoguus to bag of words. Graph2Vec then employs a skipgram model with negative sampling to learn a vector representation for each graph by maximizing the likelihood of observing its constituent subgraphs. So, graphs sharing similar local structural patterns are embedded closer in the learned representation space. Graph2Vec doesn't require node attributes and is fully unsupervised, making it suitable for datasets with limited or noisy labels.

B. NetLSD

NetLSD [2] represent a given graph using heat kernel signatures derived from the Laplacian Spectrum. Specifically, it computes a graph descriptor based on the trace of the heat kernel at different time scales and manages to capture both local and global structural properties. The NetLSD's algorithm is shown below:

Algorithm 3 NetLSD (Heat Trace Graph Signature)

Require: Graph $G = (V, E)$
Require: $T = \{t_1, \ldots, t_K\}$: set of time scales (log-spaced)
Require: $\mathcal{L}$: Laplacian type (*normalized* or *combinatorial*)
Require: m: number of eigenvalues to use (optional; $m = |V|$ for exact)
Ensure: Signature vector $s \in \mathbb{R}^K$ (optionally normalized)

Construct adjacency matrix A of G and degree matrix D
if $\mathcal{L}$ is normalized **then**
  $L \leftarrow I - D^{-1/2} A D^{-1/2}$
else
  $L \leftarrow D - A$
end if
Compute smallest m eigenvalues $\{\lambda_1, \ldots, \lambda_m\}$ of L
for $k \leftarrow 1$ to K **do**
  $s_k \leftarrow \sum_{i=1}^{m} \exp(-t_k \lambda_i)$ ▷ heat trace
end for
$s \leftarrow \log(s)$ ▷ optional but common (stabilizes scale)
$s \leftarrow \textsc{Normalize}(s)$ ▷ e.g., $s/\|s\|_2$ (optional)
return s

The key advantage of this method lies in its permutation invariance and scalability as we will see later. Instead of relying on explicit node mappings or subgraph enumeration, NetLSD creates a summary of graph's structure through spectral descriptors that are known to be robust in small structural variations. To improve computational efficiency, it approximates the Laplacian eigenvalue distribution using stochastic trace estimation and so it avoids full eigendecomposition. This hepls NetLSD to maintain stable performace even for large graphs.

C. GIN

The Graph Isomorphism Network (GIN) [3], known as a supervised graph neural network, is designed to maximize discriminative power among GNNs. GIN's node representation are updated by aggregating context from neighbor nodes. At each layer, node features are updated using a sum aggregation function followed by a multilayer perceptron (MLP), allowing the model to distinguish different graph structures effectively as shown in Algorithm 4. Node embeddings are then aggregated using a global pooling operation to produce a graph representation. This representation is then passed to a classifier and trained end-to-end using labeled data as shown in Algorithm 5.

In contrast to unsupervised methods, GIN learns task specific embeddings optimized directly for classification accuracy. While this leads to great performance, it requires labeled datasets, which is difficult to obtain, and needs higher computational cost due to iterative training.

Algorithm 4 GIN Forward Pass (Graph Embedding)

Require: Graph $G = (V, E)$, node features $\{h_v^{(0)}\}_{v \in V}$
Require: L: number of GIN layers
Require: $\{\epsilon^{(k)}\}_{k=1}^{L}$: learnable or fixed scalars
Require: $\{\mathrm{MLP}^{(k)}\}_{k=1}^{L}$: layer-wise MLPs
Require: READOUT: permutation-invariant pooling (sum/mean/max)
Ensure: Graph embedding z_G

for $k \leftarrow 1$ to L **do**
  for all $v \in V$ **do**
    $m_v^{(k)} \leftarrow \sum_{u \in \mathcal{N}(v)} h_u^{(k-1)}$
    $h_v^{(k)} \leftarrow \mathrm{MLP}^{(k)}\Big((1 + \epsilon^{(k)})\, h_v^{(k-1)} + m_v^{(k)}\Big)$
  end for
end for
$z_G \leftarrow \mathrm{READOUT}\big(\{h_v^{(L)}\}_{v \in V}\big)$
return z_G

Algorithm 5 Training a GIN for Graph Classification

Require: Dataset $\mathcal{D} = \{(G_i, y_i)\}_{i=1}^{N}$
Require: L: number of layers, epochs E, learning rate α, batch size B
Require: Loss function $\ell(\cdot, \cdot)$ (e.g., cross-entropy)
Ensure: Trained parameters Θ of GIN + classifier

Initialize model parameters Θ
for *epoch* $\leftarrow 1$ to E **do**
  Shuffle $\mathcal{D}$
  for each mini-batch $\mathcal{B} \subset \mathcal{D}$ of size B **do**
    for all $(G, y) \in \mathcal{B}$ **do**
      $z_G \leftarrow \textsc{GINForward}(G)$ ▷ Alg. 4
      $\hat{y} \leftarrow \textsc{Classifier}(z_G)$
    end for
    $\mathcal{L} \leftarrow \frac{1}{|\mathcal{B}|} \sum_{(G,y) \in \mathcal{B}} \ell(\hat{y}, y)$
    Update $\Theta \leftarrow \Theta - \alpha \nabla_\Theta \mathcal{L}$ ▷ e.g., Adam
  end for
end for
return Θ

D. Implementation Details

IV. Datasets

The experiments in this work are conducted on three widely used benchmark datasets from the TUDataset collection: MUTAG, ENZYMES, and IMDB-MULTI. These datasets represent diverse application domains and exhibit multiple graph sizes, densities and availability of node attributes, making them suitable for evaluating graph embedding methods.

A. MUTAG

MUTAG [4] is a molecular graph dataset consisting of chemical compounds where each graph represents a molecule.Nodes correspond to atoms and edges represent chemical bonds between atoms. The task is to classify molecules into two classes according to their mutagenic

effect on a bacterium. The dataset contains a small number of graphs with discrete node labels representing atom types. Graphs in MUTAG are relatively small and sparse, making the dataset a common benchmark for evaluating graph classification methods on molecular data with labeled nodes.

B. ENZYMES

The ENZYMES [5] dataset is derived from protein tertiary structures and contains graphs representing enzymes. Nodes correspond to amino acids and edges indicate spatial proximity or interactions between residues. Each graph is labeled according to the enzyme's functional class resulting in a multi class classification task.

C. IMDB-MULTI

MDB-MULTI [5] is a social network dataset constructed from movie collaboration data. Each graph represents the ego network of actors appearing in a movie, nodes correspond to actors and edges indicate coappearance relationships. Unlike molecular datasets, IMDB-MULTI does not provide node labels or attributes.

V. Experiments and Evaluation

A. Classification

SVM/MLP classifier for embeddings.

Report accuracy, F1-score, AUC.

Computational metrics: training time, generation time, memory usage.

Analysis of embedding dimensionality vs. performance.

B. Clustering

k-means/spectral clustering.

ARI scores and qualitative visualization (t-SNE or UMAP).

Discussion of cluster separation quality.

C. Stability Analysis

Random perturbations (edges/nodes).

Compare embedding changes and drop in classification accuracy.

VI. Results

Tables and figures summarizing all tasks.

Highlight trade-offs: performance vs. efficiency vs. stability.

References

[1] A. Narayanan, M. Chandramohan, L. Chen, Y. Liu, and S. Saminathan, "graph2vec: Learning Distributed Representations of Graphs," *arXiv preprint arXiv:1707.05005*, 2017.

[2] A. Tsitsulin, D. Mottin, P. Karras, A. Bronstein, and E. Müller, "NetLSD: Hearing the Shape of a Graph," in *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '18)*, 2018.

[3] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How Powerful are Graph Neural Networks?" in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2019.

[4] A. K. Debnath, R. L. López de Compadre, G. Debnath, A. J. Shusterman, and C. Hansch, "Structure–activity relationship of mutagenic aromatic and heteroaromatic nitro compounds. Correlation with molecular orbital energies and hydrophobicity," *Journal of Medicinal Chemistry*, vol. 34, no. 2, pp. 786–797, 1991, doi: 10.1021/jm00106a046.

[5] R. A. Rossi and N. K. Ahmed, "The Network Data Repository with Interactive Graph Analytics and Visualization," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2015.